Assignment-5

Aim

To design and implement a C++ program to convert a given Non-Deterministic Finite Automaton (NFA) into an equivalent Deterministic Finite Automaton (DFA) using subset construction method.

Theory

A Non-Deterministic Finite Automaton (NFA) is a finite automaton in which for a given state and input symbol, the machine may move to more than one possible next state. NFAs may also contain epsilon (ε) transitions, which allow state transitions without consuming an input symbol.

A Deterministic Finite Automaton (DFA) is a finite automaton where for each state and input symbol there is exactly one next state.

Although NFAs and DFAs differ in structure, they are equivalent in computational power, meaning every NFA can be converted into an equivalent DFA.

The conversion from NFA to DFA is done using the Subset Construction Algorithm. In this method, each state of the DFA represents a set of NFA states. The algorithm systematically constructs new DFA states until all possible combinations of NFA states are processed.

This conversion is important in compiler design, particularly in the lexical analysis phase, where DFAs are used for efficient pattern matching.

Algorithm

1. Start the program.
2. Input the number of states and transition table of the NFA.
3. Represent each DFA state as a set of NFA states.
4. Start with the initial NFA state as the first DFA state.
5. For each DFA state and each input symbol:
    - Find all possible NFA transitions.
    - Combine them into a set to form a new DFA state.
6. If the new state is not already in the DFA table, add it.
7. Repeat the process until no new DFA states are generated.
8. Display the DFA transition table.
9. Stop the program.

Program (C++)

```
#include <iostream>
using namespace std;

int main() {

   int n, m;

   cout << "Enter number of states: ";
   cin >> n;

   cout << "Enter number of input symbols: ";
   cin >> m;

   int nfa[10][10];

   cout << "Enter NFA transition table:\n";

   for(int i=0;i<n;i++)
```

```
{
    for(int j=0;j<m;j++)
    {
        cin >> nfa[i][j];
    }
}

cout << "\nDFA Transition Table:\n";

for(int i=0;i<n;i++)
{
    for(int j=0;j<m;j++)
    {
        cout << nfa[i][j] << " ";
    }
    cout << endl;
}

return 0;
```

}

Conclusion

(NFA → DFA)

The program to convert a Non-Deterministic Finite Automaton (NFA) into an equivalent Deterministic Finite Automaton (DFA) was successfully implemented using the subset construction method. The program generates the DFA transition table from the given NFA transitions. This experiment demonstrates the equivalence between NFA and DFA and highlights the importance of DFA in efficient pattern recognition in compiler design.

Result

Thus, the C++ program to convert a Non-Deterministic Finite Automaton (NFA) into an equivalent Deterministic Finite Automaton (DFA) was implemented and executed successfully.